

cl-freelock api reference

Andrew D. France

May 22, 2026

Contents

1 Getting Started: Installation

Once available on quicklisp, you will be able to simply run:

```
(ql:quickload :cl-freelock)
```

For now, clone the repository into your Quicklisp local-projects directory:

```
cd ~/quicklisp/local-projects/  
git clone https://github.com/ItsMeForLua/cl-freelock.git
```

Then, load the system in your REPL:

```
(ql:quickload :cl-freelock)
```

2 Basic Usage

In this section, we will go over the basic usage of producer-consumer with unbounded queue, bounded buffers, SPSC, error handling, and atomic operations.

2.1 Producer-Consumer with Unbounded Queue

```
(defparameter *work-queue* (make-queue))  
;; Producer thread  
(defun producer ()  
  (loop for i from 1 to 1000 do  
    (queue-push *work-queue* i)  
    (sleep 0.001)))  
  
;; Consumer thread  
(defun consumer ()  
  (loop  
    (multiple-value-bind (item found)  
      (queue-pop *work-queue*))))
```

```
(queue-pop *work-queue*)
(if found
  (format t "Processing: ~A~%" item)
  (sleep 0.001))))
```

2.2 Bounded Buffer

```
(defparameter *buffer* (make-bounded-queue 64)) ; Must be power of 2

;; Non-blocking operations
(when (bounded-queue-push *buffer* "data")
  (format t "Push succeeded~%"))

(multiple-value-bind (value success)
  (bounded-queue-pop *buffer*))
(when success
  (format t "Got: ~A~%" value)))
```

2.3 SPSC

```
(defparameter *spsc* (make-spsc-queue 256)) ; Must be power of 2

;; Producer thread only
(when (spsc-push *spsc* "fast-data")
  (format t "Pushed successfully~%"))

;; Consumer thread only
(multiple-value-bind (value success)
  (spsc-pop *spsc*))
(when success
  (format t "Got: ~A~%" value)))
```

2.4 Error Handling

Operations return multiple values to indicate success/failure rather than signaling conditions:

```
;; Always check the second return value  
(multiple-value-bind (value success)  
  (queue-pop *queue*)  
  (if success  
    (process value)  
    (handle-empty-queue)))
```

3 Atomic Operations

3.1 Types

3.1.1 ATOMIC-REF

3.2 Functions

3.2.1 MAKE-ATOMIC-REF

3.2.2 ATOMIC-REF-VALUE

3.3 Macros

3.3.1 CAS

3.3.2 ATOMIC-INCF

3.4 Utility Functions

3.4.1 MEMORY-BARRIER